Year Up United
Data Analyst Training Academy: Python Tools for Data Analysis

Week 7 Student Guide

This courseware is to be distributed to accompany instructor led training.

Table of Contents

Week 7 Learning Objectives

- This week you will learn:
  - ◊ Creating, indexing, reshaping, and performing operations on NumPy arrays (1D and 2D)
  - ◊ Using NumPy's random module to generate sample datasets for testing and simulation
  - ◊ NumPy mathematical and statistical functions: sum, mean, std, round, where, and sort
  - ◊ Creating pandas Series and DataFrames from dictionaries, arrays, and CSV/Excel files
  - ◊ Exploring a dataset using .head(), .tail(), .info(), .describe(), and .value_counts()
  - ◊ Cleaning data using .dropna(), .fillna(), .replace(), .drop(), and .drop_duplicates()
  - ◊ Transforming and summarizing data using .sort_values(), .groupby(), .rank(), and .merge()
  - ◊ Producing and customizing data visualizations using Matplotlib: line, bar, scatter, histogram, pie, and multi-panel subplots
  - ◊ Chart formatting: labels, titles, gridlines, markers, colors, figure size, and saving to file
  - ◊ Building a linear regression model with scikit-learn: defining features and target, splitting train/test data, fitting the model, generating predictions, evaluating with R^2, and visualizing actual vs. predicted
  - ◊ Interpreting regression results: what slope, intercept, and R^2 communicate about the data

Week 7 Learning Outcomes

- By the end of this week you will be able to:
  - ◊ Load a CSV file into a pandas DataFrame and produce a summary report (row count, column types, basic statistics, value counts for a categorical column)
  - ◊ Clean a DataFrame with missing values and export the result to a new CSV file
  - ◊ Produce a labeled, titled, appropriately sized chart from a DataFrame and save it as a .png
  - ◊ Run the full linear regression workflow on a given dataset and explain in plain language what the model's R^2 score means
  - ◊ Describe the role of each of the four libraries (NumPy, pandas, Matplotlib, scikit-learn) and when to use each one

Module 1

Continuing NumPy

Section 1.A

Continuing NumPy

1.A.1 Using loops with arrays

- Loops can be used to iterate over one- or multi-dimensional arrays
  - ◊ 1D array – loops through each element one by one
  - ◊ N-dimensional array –
    - * First level loop runs against outermost dimension; each nested loop runs against next inner dimension
    - * Example:

```
arr3d = np.array([[[8,7], [6,5]], [[4,3], [2,1]]])
```

```
for x in arr3d:
    print(x)
```

output:

```
[[8 7]
 [6 5]]
[[4 3]
 [2 1]]
```

```
for x in arr3d:
    for y in x:
        print(y)
```

output:

```
[8 7]
[6 5]
[4 3]
[2 1]
```

```
for x in arr3d:
    for y in x:
        for z in y:
            print(z)
```

output:

```
8
7
6
5
4
3
2
1
```

1.A.2 Joining arrays with `concatenate()`

- Joining puts the contents of two or more arrays into a single array
  - ◊ In NumPy, arrays are joined on an axis
  - ◊ The sequence of arrays to be joined is passed to the `concatenate()` function, along with the desired axis (if not specified, defaults to 0 axis)
  - ◊ Examples:
    - * Joining two 1D arrays

```
arr_a = np.array([1, 2, 3])
arr_b = np.array([4, 5, 6])
arr_c = np.concatenate((arr_a, arr_b))
print(arr_c)                                    # [1, 2, 3, 4, 5, 6]
```

    - * Joining two 2D arrays on axis 1 (i.e. columns)

```
arr_d = np.array([[1, 2], [3, 4]])
arr_e = np.array([[5, 6], [7, 8]])
arr_f = np.concatenate((arr_d, arr_e), axis=1)
print(arr_f)                                    # [[1, 2, 5, 6],
                                                #  [3, 4, 7, 8]]
```

- Arrays can also be joined on a *new* axis using `stack()`

```
arr_g = np.array([1, 2, 3])
arr_h = np.array([4, 5, 6])
arr_i = np.stack((arr_g, arr_h))
print(arr_i)                                    # [[1, 2, 3],
                                                #  [4, 5, 6]]

arr_j = np.stack((arr_g, arr_h), axis=1)
print(arr_j)                                    # [[1, 4],
                                                #  [2, 5],
                                                #  [3, 6]]
```

1.A.3 NaN values

- In NumPy and pandas, “null” values are translated to **`NaN`**
  - ◊ `NaN` stands for “not a number” but is broadly used as a placeholder for missing data
- `NaN` and `None` are both types of NA values, however…
- … `NaN` is *not* equivalent to `None` (and neither is equivalent to zero)
  - ◊ `None` means “the value here is set to no value”
  - ◊ `NaN` means “the value here is missing”
  - ◊ `0` means “the value here is a value of zero”
- From a computing perspective, another benefit of `NaN` is that it is stored with NumPy’s efficient float64 data type
  - ◊ By contrast, `None` is stored with the NumPy object dtype, which takes up more memory and also changes behavior when performing vectorized operations across arrays

1.A.4 Stacking helper functions

- NumPy includes helper functions to stack along rows/horizontal, columns/vertical, or height/depth
  - ◊ For each of the below examples, we'll start with the same base arrays:

```
arr1 = np.array([1, 2, 3])
arr2 = np.array([4, 5, 6])
```

    - * **`hstack()`** – stack horizontally along column axis

```
arr = np.hstack((arr1, arr2))
print(arr)                          # [1 2 3 4 5 6]
```

    - * **`vstack()`** – stack vertically along row axis

```
arr = np.vstack((arr1, arr2))
print(arr)                          # [[1, 2, 3],
                                    #  [4, 5, 6]]
```

    - * **`dstack()`** – stack along height/depth

```
arr = np.dstack((arr1, arr2))
print(arr)                          # [[[1, 4],
                                    #   [2, 5],
                                    #   [3, 6]]]
```

1.A.5 Splitting arrays

- Split is the reverse of join – splitting breaks one array into multiple
  - ◊ The function used is **`array_split()`**, passing arguments of the array to be split and the number of splits, with the result as a list of arrays
  - ◊ If the starting array cannot be split evenly among the new arrays, the results will adjust from the end accordingly
    - * Examples, starting with the same array:
      ```
      arr = np.array([1, 2, 3, 4, 5, 6])
      ```
    - * Ex. 1: Split into equally-sized arrays
      ```
      newarr1 = np.array_split(arr, 3)
      print(newarr1) # [array([1, 2]), array([3, 4]), array([5, 6])]
      ```
    - * Example 2: Split does not divide evenly into starting array
      ```
      newarr2 = np.array_split(arr, 4)
      print(newarr2) # [array([1, 2]), array([3, 4]), array([5]), array([6])]
      ```

- The same syntax applies to splitting multi-dimensional arrays; additionally, you can specify which axis to split around

```
arr=np.array([[1, 2, 3], [4, 5, 6], [7, 8, 9], [10, 11, 12]])
newarr = np.array_split(arr, 2, axis=0) |
print(newarr)   # [array([[1, 2, 3],
                      [4, 5, 6]]),
                 array([[ 7,  8,  9],
                      [10, 11, 12]])]
```

- Also may use helper functions *hsplit()*, *vsplit()*, *dsplit()*

1.A.6 Using `where()` to search and alter arrays

- The **`where()`** function is used to retrieve the indices of elements in an ndarray where a given condition is true
  - ◊ Syntax:
    `np.where(condition[, x, y])`
  - ◊ A condition must be specified when using where, but the next two parameters are optional (must use neither or both, not just one):
    - parameter *x* – specifies a replacement value for first condition
    - parameter y – provides a replacement value where condition was not met; effectively an "else" statement
  - ◊ If only one condition provided as argument, returns a tuple with index location where the value is found; can convert to array using `array()`

```
a = np.array([4,9,4,8,0,6])
b = np.where(a < 5)
print(b)     # (array([0, 2, 4], dtype=int64),)
print(np.array(b)) # [[0 2 4]]
```

    - This is also equivalent to NumPy's `.nonzero()` function
  - ◊ The additional parameters can be used to specify a replacement value where the condition is met:

```
c = np.where(a < 5, a, a * .9)
print(c)     # [4. , 8.1, 4. , 7.2, 0. , 5.4]

d = np.where(a < 5, 'little', a)
print(d)     # ['little', '9', 'little', '8', 'little', '6']
```

1.A.7 Sorting arrays

- NumPy's **`sort()`** function will return a copy of an array as an ordered sequence
  - ◊ Example: sorting an array of integers

```
a = np.array([14,9,4,18,0,6])
print(np.sort(a))      # [ 0,  4,  6,  9, 14, 18]
```

  - ◊ Example: sorting an array of strings

```
b = np.array(['14','9','4','18','0','6'])
print(np.sort(b))      # ['0', '14', '18', '4', '6', '9']
```

- Using `sort()` on a multi-dimensional array will sort each component array
  - ◊ Example: sorting a 3D array

```
y=np.array([[[6,2],
             [6,4]],
            [[3,1],
             [9,8]],
            [[9,0],
             [4,2]],
            [[5,4],
             [8,6]]])
print(np.sort(y))      # [[[2, 6],
                         [4, 6]],
                        [[1, 3],
                         [8, 9]],
                        [[0, 9],
                         [2, 4]],
                        [[4, 5],
                         [6, 8]]]
```

1.A.8 Filtering arrays

- Arrays can be **filtered** based on a Boolean index list / using conditional logic
  - ◊ Filtering an array means retrieving just the values that meet the condition
  - ◊ Using the standard Python library, we *could* do this with a loop:
    - * Example:

```
a = np.array([14,9,4,18,0,6])

filter_a = []

for x in a:
    if x > 10:
       filter_a.append(True)
    else:
       filter_a.append(False)

b = a[filter_a]

print(b)   # [14, 18]
```

  - ◊ But NumPy has a much more efficient option!
    - * Utilizing NumPy array properties:

```
# using same starting array a

x = a > 10
a_over_10 = a[x]

print(a_over_10) # [14, 18]
```

Section 1.B

Random in NumPy

1.B.1 Random module of NumPy

- The numpy.random module implements **pseudo-random number generators** (PRNGs or RNGs, for short) which can sample on a variety of probability distributions

- Refresher: What is randomness?
  - ◊ Generating a random number does *not* mean you'll get a different number every time
    - * If you roll a die 4 times, each set of results below is equally (un)likely:

      1, 2, 3, 4 1, 1, 1, 1 3, 1, 6, 2
  - ◊ Random means something that **cannot be predicted logically**
    - * Pseudo random: Numbers generated by a computer program are based on some kind of algorithm that generates the number, which means that the number can be predicted with the right information – therefore these are not *truly* random numbers
    - * True random: An external data source can be used to create true randomness, such as a user's mouse movements or keystrokes, or even things like measuring atmospheric noise or radioactive decay

- NumPy's random module is not suitable for applications requiring true randomness, but for our purposes it's generally sufficient
  - ◊ Applications requiring true randomness include security encryption or where randomness is foundational to the application (like a digital roulette wheel)

1.B.2 Simple random number functions

- **`randint()`** generates a random integer
  - ◊ Optional start and stop parameters allow you to specify a range; if only one number is provided, start defaults to 0
  - ◊ The *size=()* parameter is used to specify number of elements by row and column when generating an array of random integers
  - ◊ Example:

```
r_i=np.random.randint(90, 100, size=(2, 5))

print(r_i)      #   [[97 99 94 96 97]
                    [91 92 92 97 94]]
```

- **`rand()`** generates a random float between 0 and 1
  - ◊ The rand() method also allows you to specify the shape of the array, but the syntax is a little different
  - ◊ Because rand() does not accept start/stop parameters, it also does not use the size=() label
  - ◊ Instead, array size is specified as: rand([number rows, number columns]) – if only one number is provided, number of rows defaults to 1
  - ◊ Example:

```
r_f=np.random.rand(3, 2)

print(r_f)      #   [[0.31242925   0.92632055]
                    [0.68851952   0.61750025]
                    [0.45294665   0.46720157]]
```

1.B.3 Random from array using `choice()`

- The **`choice()`** method allows you to generate one random value based on a 1D array **or** to return an array of random values based on a 1D array
  - ◊ Again, optional `size=()` parameter can be used to specify how many rows and columns the resulting array should have
  - ◊ Example 1:

```
a = np.random.choice([3, 5, 7, 9], size=(3, 5))

print(a)    #   [[9   9   7   9   5]
                 [9   7   5   9   7]
                 [5   3   9   5   9]]
```

  - ◊ Example 2:

```
b = np.array([414, 715, 651, 612, 630])

c = np.random.choice(b, size=(5))
print(c)    #   [630 414  715  715  651]

d = np.random.choice(b)
print(d)    #   651
```

1.B.4 Random distributions

- First off, the core concept of *data distribution* refers to how often each value in a list occurs – a valuable concept in data analysis!
  - ◊ The random module provides useful methods to return different kinds of randomly generated data distributions, which is very helpful when creating sample/practice data

- A **random distribution** is a set of random numbers that follow a certain *probability density function*
  - ◊ Probability Density Function = function that describes a continuous probability, i.e. probability of all values in an array

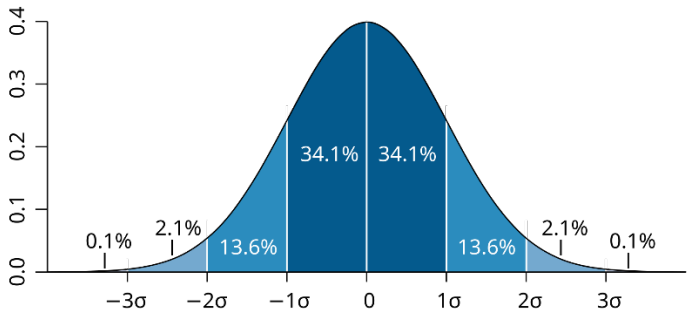
- We can use `choice()` to generate an array with a random distribution based on defined probabilities
  - ◊ Probability (parameter `p=[]`) can be specified for each value in the source array, where probability is a value between 0 (never occurs) and 1 (always occurs); total of all probability values must equal 1
  - ◊ Example

```
x= np.random.choice(
    ['cat','dog','bunny','snake','fish'],
     p=[0.2, 0.1, 0.6, 0.0, 0.1], size=(10))

print(x) # ['bunny' 'bunny' 'cat' 'bunny' 'bunny' 'dog' 'dog' 'bunny' 'bunny' 'bunny']
```

    - * Notice in this example: even though ‘cat’ is more likely to occur than ‘dog’, it’s still possible to get more occurrences of ‘dog’ than ‘cat’ (however, ‘snake’ will *never* occur, as it has `p` value of zero)

1.B.5 Normal distribution

- The **normal distribution**, also known as Gaussian distribution or the "bell curve", fits the probability distribution of many different types of real-life events



For the normal distribution, 68.27% of the values in the set occur less than one standard deviation from the mean; 95.45% within two standard deviations from the mean; and 99.73% within three standard deviations.

Image by M. W. Toews, via Wikipedia commons.wikimedia.org/w/index.php?curid=1903871

- NumPy's random module includes the method **`normal()`** to generate a normal data distribution, using three parameters:
  - ◊ **`loc`** – the mean, or the peak of the bell
  - ◊ **`scale`** – standard deviation, i.e. spread/width of the distribution
  - ◊ **`size`** – the shape of the returned array (array length or rows, columns)
  - ◊ Example:

```
x = np.random.normal(loc=5, scale=2, size=(30))

print(x)    #   [ 7.31969953  5.17885056  4.62581317  3.33685624  4.86686884  6.53762077
                  4.68502329  5.28155977  2.96170333  3.6019154   3.55382812  5.1687159
                 10.5534121   1.1960087   3.10580747  2.89190108  4.86526659  6.07605373
                  2.60980551  4.04195524  6.14698288  6.55377346  4.32039062  4.78897072
                  4.28304723  4.78772225  3.95004663  5.15332982  5.58148874  6.60058118]
```

- There are other distribution types that can be generated with NumPy: binomial, uniform, Pareto (aka 80/20 rule)...

Section 1.C

Mathematical Tools in NumPy

1.C.1 Permutations of arrays

- A **permutation** is a different arrangement / order of the same set of values
  - ◊ For example, [1, 2] has only one permutation: [2, 1]
  - ◊ The more values in a set, the more permutations, e.g. [1, 2, 3] has permutations of [1, 3, 2], [2, 1, 3], [2, 3, 1], [3, 1, 2] and [3, 2, 1]

- With NumPy, permutations of an array can be generated using either shuffle() or permutation()
  - ◊ shuffle() changes the order of elements in-place, i.e. reorders the array
    - * Example:

```
h = np.array([1, 2, 3, 4, 5])

np.random.shuffle(h)

print(h)   #   [1 4 2 3 5]
```

  - ◊ permutation() returns a new array with the elements in a new random order
    - * Example:

```
i = np.array([1, 2, 3, 4, 5])

print(np.random.permutation(i)) #   [4 3 2 1 5]

print(i)   #   [1 2 3 4 5]
```

1.C.2 NumPy ufuncs

- A **ufunc** is a *universal function* in NumPy that operates on arrays
  - ◊ There are currently more than 60 universal functions defined in NumPy on one or more types, covering a wide variety of operations including math operations like `add()`, `divide()`, `rint()`; comparison functions like `greater()`, `greater_equal()`, and more
  - ◊ Ufuncs are used to implement **vectorization** in NumPy, converting iterative statements into a vector based operation (faster performance than iterating over elements)
    - * For instance, in standard Python we could add the elements of two lists by using a for loop with the `zip()` method to iterate over each element and add them to a new list…
    - * …but NumPy has a more efficient ufunc, **`add()`**, that accepts the two list names as arguments and performs the calculation as a vector-based operation

- Ufuncs take additional arguments, such as:
  - ◊ `where` – Boolean array or condition defining where the operations should take place
  - ◊ `dtype` – defines the return type of elements
  - ◊ `out` – names output array where the return value should be copied

- You can create your own ufunc by defining a function, then adding it to your NumPy ufunc library with the `frompyfunc()` method

- For more detail, see ufunc documentation at: https://numpy.org/doc/stable/reference/ufuncs.html

1.C.3 Basic arithmetic with arrays

- While arithmetic operators (`+ - * /`) do work to perform operations on values between arrays, you can also use NumPy functions to perform math on arrays (or array-like objects) with the option of specifying a `where` condition

- Arithmetic functions (require 2 input args of arrays/lists/tuples):
  - ◊ add() – first array values plus second array values
  - ◊ subtract() – first array values minus second array values
  - ◊ multiply() – first array values times second array values
  - ◊ divide() – first array values divided by second array values
  - ◊ power() – takes the value from the first array and raises it to the power of the value from the second array
  - ◊ mod() or remainder() – returns remainder when dividing values of first array by values of second array
  - ◊ divmod() – performs division and returns two arrays, with the first containing the quotient of first array divided by second array, and the second array containing the remainder
  - ◊ absolute() or abs() – accepts 1 argument of a single array and returns absolute value of the values in that array

1.C.4 Rounding decimal numbers in NumPy

- NumPy offers five main methods of rounding decimals:
  - ◊ trunc() returns the number with the decimal places chopped off
    - * Additional arguments may be provided for numpy.trunc – see numpy.org/doc/stable/reference/generated/numpy.trunc.html
  - ◊ fix() – rounds to the nearest integer toward zero
  - ◊ floor() – rounds off to the nearest lower integer
  - ◊ ceil() – rounds off to the nearest higher integer
  - ◊ round() – must specify number of decimal places; generally follows conventional rule of rounding, except that for values exactly halfway between rounded decimal values, NumPy rounds to the nearest even value
    - * Example:

```
x = .123456
y = np.round(x, 4)
print(y)  #   0.1235

a = .12345
b = np.round(a, 4)
print(b)  #   0.1234
```

    - * For documentation on numpy.round, see numpy.org/doc/stable/reference/generated/numpy.round.html

1.C.5 Summations in NumPy

- While addition is done between two elements or values, summation happens over the sequence of elements

- To find the total of all elements in all arrays, use **`sum()`**

  ◊ If you would like to find the sum of all elements along a specific axis (i.e. sum each row, or sum each column), include the **`axis=`** parameter

  ◊ Example:

```
a = np.array([1, 2, 3])
b = np.array([1, 2, 3])

c = np.sum([a, b], axis=1)

print(c)    #[6 6]
```

- Cumulative sum, **`cumsum()`**, finds the rolling total of the elements in an array

  ◊ Example:

```
d = np.array([1, 2, 3])

e = np.cumsum(d)

print(e)    #[1 3 6]
```

1.C.6 Products

- To find the product of all elements in an array / arrays, use **`prod()`**
  - ◊ As with `sum()`, you can also specify an axis to find the product of all rows or all columns
  - ◊ Example:

```
a = np.array([1, 2, 3, 4])
b = np.array([5, 6, 7, 8])

c = np.prod([a, b], axis=0)

print(c)    # [ 5 12 21 32]
```

- You can also use **`cumprod()`** to find the rolling product of elements
  - ◊ Example:

```
d = np.array([1, 2, 3])

e = np.cumprod(d)

print(e)    # [1 2 6]
```

1.C.7 Differences

- A discrete difference means subtracting two successive elements:
  - ◊ E.g. for [1, 2, 3, 4], the discrete difference would be 2-1, then 3-2 then, 4-3 = [1, 1, 1]

- To find the discrete difference, use the **`diff()`** function
  - ◊ Example:

```
x = np.array([10, 5, 20, 3])

y = np.diff(x)

print(y)    #[-5  15 -17]
```

  - ◊ To perform the difference calculation over an array multiple times, use the **`n=`** parameter

```
x = np.array([10, 5, 20, 3])

y = np.diff(x, n=2)

print(y)    #[ 20  -32]
```

  - ◊ Note that the number of elements in the array is reduced by one each time the difference is calculated
    - * If you set `n=` equal to or higher than the number of elements in the array, the returned result will be an empty array

1.C.8 Other mathematical operations

- There are many, many other mathematical operations that can be performed using NumPy, so be open to checking resources and looking things up

- Some examples of other things you can do with NumPy:
  - ◊ Find the lowest common multiple or greatest common denominator of numbers in an array
  - ◊ Perform trigonometry operations
  - ◊ Perform set operations on 1D arrays, like union, intersection, difference, and symmetric difference between sets
  - ◊ Use NumPy's `.unique()` method to create each set array by ensuring its elements are unique and not repeated

Module 2

pandas

Section 2.A

pandas Basics

2.A.1 Getting started with pandas

- pandas can be installed using PIP from the command line:

```
pip install pandas
```

- Once pandas is installed, you can call it in a Python script using

```
import pandas
```

  - ◊ Often Python developers use `import pandas as pd` for efficiency

- Any time you call a function from the imported library, you must use the library name (or alias) followed by the function in dot notation
  - ◊ Example 1:

```
import pandas

a = pandas.Series([3, 217, 182])
```

  - ◊ Example 2:

```
import pandas as pd

z = pd.Series([3, 217, 182])
```

2.A.2 pandas Series

- A Series is a one-dimensional array that can hold data of any type
  - ◊ Think of a Series like a column in a table
  - ◊ Values are labeled by index number as a default, but labels can also be specified
  - ◊ Examples:

```
a_list = [3, 217, 182]
print(a_list)   # [3, 217, 182]  # just a plain Python list

a = pd.Series(a_list)
print(a)    #   0  3
            1  217
            2  182

a2 = pd.Series(a_list, index = ['a', 'b', 'c'])
print(a2)   #   a  3
            b  217
            c  182
```

- A series can also be created from a dictionary of key-value pairs
  - ◊ The keys automatically become the labels:

```
b_dict = {'a': 3, 'b': 217, 'c': 182}
b = pd.Series(b_dict)
print(b)    #   a  3
            b  217
            c  182
```

2.A.3 pandas DataFrames

- 2-dimensional data structures in pandas are known as **DataFrames**
  - ◊ Where a Series is like a column, a DataFrame is like a table
  - ◊ Series can be combined to create a DataFrame, or a DataFrame can be created from a dictionary or from other data

- Index labels/names can be specified same as with Series
  - ◊ Example:

```
data = {
    "product_num": [1658, 2476, 3911],
    "quantity": [120, 54, 98]
    }
inventory = pd.DataFrame(data)
```

  - ◊ When a named DataFrame is called using a graphic-friendly editor like JupyterNotebook, it is returned in a handsome tabular format:

	product_num	quantity
0	1658	120
1	2476	54
2	3911	98

- To return the entire DataFrame contents as a tab-delimited string, use the method `.to_string()`

2.A.4 Creating a DataFrame

- A DataFrame can be created from a Python collection like a list or tuple, a NumPy array or pandas Series, or using tabular data imported from a file
  - ◊ Each of the following one-dimensional examples:

```
df01 = pd.DataFrame((1,2,3,4))

arr = np.array([1,2,3,4])
df02 = pd.DataFrame(arr)

ser = pd.Series(arr)
df03 = pd.DataFrame(ser)
```

    - * ... would return a DataFrame that looks like this:

	0
0	1
1	2
2	3
3	4

  - ◊ A DataFrame from a two-dimensional array has columns *and* rows:

```
arr2 = np.array([[1,2,3,4]])
df02 = pd.DataFrame(arr2)
df02
```

	0	1	2	3
0	1	2	3	4

2.A.5 Inserting columns into a DataFrame

- Additional column data can be added using the **`.insert()`** method, specifying the index position of the column, new column name, and source/values
  - ◊ Example

```
df01 = pd.DataFrame((1,2,3,4))
df01.insert(1, 1, [20,20,40,50])
df01
```

	0	1
0	1	20
1	2	20
2	3	40
3	4	50

2.A.6 NumPy sampling in a DataFrame

- To generate a sample DataFrame, NumPy's **`.arange()`** function can be used to generate a number range to populate values, along with **`.reshape()`** to specify how to arrange data into rows and columns
  - ◊ Example

```
sampledata = pd.DataFrame(np.arange(8).reshape((2,4)),
                          index=['apple','banana'],
                          columns=['a','b','c','d'])

print(sampledata)      #            a    b    c    d
                         apple      0    1    2    3
                         banana     4    5    6    7
```

2.A.7 Indexing of DataFrames

- By default, rows in a DataFrame are indexed starting with 0

- As with a Series, labels can be added to a DataFrame index

  ◊ Example:

```
rowdata = {'col1': [715, 651], 'col2': [414, 612]}
areacodes = pd.DataFrame(rowdata, index = ['WI', 'MN'])

print(areacodes)                    #      col1  col2
                                    WI     715   414
                                    MN     651   612
```

- Row data can be retrieved by slicing the index

  ◊ Example:

```
print(areacodes['MN':])     #      col1  col2
                            MN     651   612
```

2.A.8 Accessing rows by `.loc[]` and `.iloc[]`

- You can use the **`.loc[]`** attribute to access a set of rows by index

```
data = {
    "product_num": [1658, 2476, 3911],
    "quantity": [120, 54, 98]
    }
inventory = pd.DataFrame(data)
```

  - ◊ A single row (here, row 0) is returned as a Series

```
inventory.loc[0] #    product_num     1658
                 quantity     120
                 Name: 0, dtype: int64
```

  - ◊ Multiple rows (here, rows [1,2]) are returned as a DataFrame

```
inventory.loc[1,2]
```

	product_num	quantity
1	2476	54
2	3911	98

  - ◊ A *new* row can also be assigned to a DataFrame using `.loc`:

```
inventory.loc[3] = [7878, 0]
inventory
```

	product_num	quantity
0	1658	120
1	2476	54
2	3911	98
3	7878	0

- If the DataFrame uses a named index, the integer index can still be accessed using **`.iloc[]`**

2.A.9 Reading files into a DataFrame

- If a dataset is stored in a file such as a .csv, Excel file, or .json, pandas can be used to import the data into a DataFrame
  - ◊ Examples:

```
import pandas as pd   # First step is always to import pandas!

df1 = pd.read_csv('sample_data.csv')

df2 = pd.read_excel('sample_Excel_data.xlsx')

df3 = pd.read_json('sample_json.json')
```

  - ◊ If the resulting DataFrame contains *many* rows, on print only the headers plus first 5 and last 5 rows will be returned
    - * You can check the default max number of rows to be returned using the `pd.options.display.max_rows` statement and can also change the same setting by setting that statement to `= [new max number]`

- You can also write a DataFrame to a file (i.e. export data)
  - ◊ Examples:

```
df1.to_csv('filename.csv', index=False)

df.to_excel('filename.xlsx', index=False)

df.to_json('filename.json')
```

2.A.10 Generating info about the DataFrame

- The DataFrame object carries some useful info methods
  - ◊ To preview only the first / last records from a DataFrame:
    - * **`.head()`** displays the header and first *n* rows (5 by default)
    - * **`.tail()`** displays the last *n* rows (5 by default)
  - ◊ To see statistics on numeric columns, use **`.describe()`**:

```
* sales_det = pd.DataFrame({
      "prod_num": [658, 2476, 658, 2476, 3911, 3911],
      "price": [7.89, 5.49, 7.89, 5.49, 6.25, 6.25]})
```

```
sales_det
```

	prod_num	price
0	658	7.89
1	2476	5.49
2	658	7.89
3	2476	5.49
4	3911	6.25
5	3911	6.25

```
sales_det.describe()
```

	prod_num	price
count	6.000000	6.000000
mean	2348.333333	6.543333
std	1458.143020	1.097099
min	658.000000	5.490000
25%	1112.500000	5.680000
50%	2476.000000	6.250000
75%	3552.250000	7.480000
max	3911.000000	7.890000

2.A.11 More tools to explore the DataFrame

- Use **`.info()`** to see summary info including data types by column

- To see a frequency count of each unique value in a column, use **`.value_counts()`**

  ◊ This is especially useful for categorical columns, as it tells you how many times each category appears in the data

  ◊ Example (continuing with `sales_det` from previous page):

  ```
  sales_det['prod_num'].value_counts()
  ```

  Result:
  prod_num
  658 2
  2476 2
  3911 2

  ◊ By default, results are sorted from most to least frequent

  ◊ `.value_counts()` is one of the most practical tools for exploratory data analysis because it quickly answers "what's in this column, and how often?"

Section 2.B

Cleaning data with pandas

2.B.1 Removing rows or columns

- To drop records from a DataFrame:
  - **`drop()`**
    - axis = 0 refers to rows
    - axis = 1 refers to columns
  - **`dropna()`** removes rows containing NaN values
- For identifying and dealing with duplicate / nonunique records:
  - **`duplicated()`** identifies duplicates (returns True for duplicate, otherwise False)
  - To remove duplicate records, use **`drop_duplicates()`** method
    - Example:

```
example_df.drop_duplicates()
```

- Note: For the above functions and many others, by default the method does not return a new DataFrame (`inplace = False`)
  - The optional argument `inplace = True` will *not* return a new DataFrame, but instead will modify the original DataFrame (making the change “in place”)
    - Example:

```
example_df.drop_duplicates(inplace = True)
```

2.B.2 Filter rows

- As with NumPy arrays, conditional logic can be used to **filter rows** in a DataFrame and add them to a new DataFrame
  - ◊ The Boolean index list / conditional logic is provided in brackets following the name of the DataFrame being filtered
    - * The conditional logic might also include an index reference of its own
  - ◊ Example:

```
inventory = pd.DataFrame({
    "product_num": [1658, 2476, 3911],
    "quantity": [120, 54, 98]})
print(inventory)   #       product_num   quantity
        0       1658     120
        1       2476     54
        2       3911     98

inv_filt = inventory[(inventory['quantity'] < 100)]
print(inv_filt)    #       product_num   quantity
        1       2476     54
        2       3911     98
```

2.B.3 Replace values in a DataFrame

- The **`replace()`** method replaces specified values with new values
  - ◊ Syntax:

```
df_name.replace(to_replace, value, [inplace=False])
```

  - ◊ Example:

```
inventory.replace({'prod_num':{1658: 658}}, inplace=True)
```

  - ◊ See documentation for replace() at https://pandas.pydata.org/pandas-docs/stable/reference/api/pandas.DataFrame.replace.html

- To replace NaN values, use **`fillna()`**
  - ◊ Example, for all NaN values in the DataFrame:

```
inventory = inventory.fillna(130)
```

  - ◊ Example, for just NaN values in a specified column:

```
inventory['prod_num'] = inventory.prod_num.fillna(130)
```

2.B.4 Fixing bad data with `.loc[]` or `.drop[]`

- Sometimes you want to fix values in specific records, or loop through a DataFrame to find and fix specific records
  - ◊ Starting with:

```
inventory = pd.DataFrame({
    "product_num": [1658, 2476, 3911],
    "quantity": [120, 54, 98]})
```

  - ◊ Using `.loc[]` to assign a new value:

```
inventory.loc[2, "quantity"] = 100
```

  - ◊ Using `.loc[]` and `.index` attribute to loop through rows and assign new values based on a condition:

```
for x in inventory.index:
    if inventory.loc[x, "quantity"] < 100:
        inventory.loc[x, "quantity"] = 100
```

- Alternatively, you might not want to attempt to fill in or fix bad data
  - ◊ For instance, maybe you cannot safely make assumptions about the intended value or the appropriate replacement value
  - ◊ In these cases, you might choose instead to **drop** records with bad data

```
for x in inventory.index:
    if inventory.loc[x, "quantity"] < 100:
        inventory.drop(x, inplace = True)
```

2.B.5 Retrieving & changing column data

- Columns can be accessed using dictionary-like notation or by using dot attribute notation

  ◊ Example 1 – Column name written in style of a key name:

```
rowdata = {'col1': [715, 651], 'col2': [414, 612]}
areacodes = pd.DataFrame(rowdata, index = ['WI', 'MN'])

print(areacodes['col1'])      #   WI    715
                                  MN    651
                                  Name: col1, dtype: int64
```

  ◊ Example 2 – Column name referenced using dot notation:

```
# using same DataFrame as above

print(areacodes.col1)         #   WI    715
                                  MN    651
                                  Name: col1, dtype: int64
```

- Columns can be added, or column values updated, by using the assignment operator (=)

  ◊ Example:

```
# using same DataFrame as above

areacodes['col3'] = [262, 218]
print(areacodes)              #        col1  col2  col3
                              WI       715   414   262
                              MN       651   612   218
```

Section 2.C

Working with data in pandas

2.C.1 Basic aggregations

- Common aggregation functions include:
  - ◊ `sum()`
  - ◊ `count()`
  - ◊ `mean()` aka average
  - ◊ `median()`
  - ◊ `min()` and `max()` (useful for identifying outliers)
- `.aggregate()` method allows multiple aggregations to be performed at once, and can specify columns and functions to perform

2.C.2 Using `groupby()`

- The **`groupby()`** function reshapes a DataFrame into groups based on specified criteria, and involves some combination of splitting the object, applying a function, and combining the results
  - ◊ This can be used to group large amounts of data and compute operations on these groups
  - ◊ Example:

```
sales_det = pd.DataFrame({
    "prod_num": [658, 2476, 658, 2476, 3911, 3911],
    "price": [7.89, 5.49, 7.89, 5.49, 6.25, 6.25]})
print(sales_det) #      prod_num  price
                    0   658       7.89
                    1   2476      5.49
                    2   658       7.89
                    3   2476      5.49
                    4   3911      6.25
                    5   3911      6.25

sales = sales_det.groupby(['prod_num']).mean()
print(sales)     #      prod_num  price
                        658       7.89
                        2476      5.49
                        3911      6.25
```

- For detailed documentation on groupby(), see:
  - ◊ https://pandas.pydata.org/pandas-docs/stable/reference/api/pandas.DataFrame.groupby.html

2.C.3 Sorting data in a DataFrame

- To sort lexicographically by row or column label, you can use the **`.sort_index()`** method which returns a new sorted object
  - ◊ Example

```
sampledata = pd.DataFrame(np.arange(8).reshape((2,4)),
                          index=['cantaloupe','banana'],
                          columns=['d','a','b','c'])

print(sampledata)    #
                          d   a   b   c
                cantaloupe  0   1   2   3
                banana      4   5   6   7

print(sampledata.sort_index(axis='columns'))
                     #
                          a   b   c   d
                cantaloupe  1   2   3   0
                banana      5   6   7   4

print(sampledata.sort_index())
                     #
                          d   a   b   c
                banana    4   5   6   7
                cantaloupe  0   1   2   3
```

- To sort by column values, use the **`.sort_values()`** method and column name(s) by which to sort
  - ◊ Missing values are sorted to the end by default, but this can be changed to put missing values first using `na_position='first'`
- For both of the above methods, by default the data is sorted in ascending order, but this can be changed by using `ascending=False`

2.C.4 Ranking data

- The **`rank()`** method assigns each value in a Series or DataFrame a rank from 1 through the total number of valid data points
  - ◊ Example:

```
rowdata = {'col1': [414, 651], 'col2': [715, 612]}
areacodes = pd.DataFrame(rowdata, index = ['WI', 'MN'])
areacodes['col3'] = [262, 218]
print(areacodes)          #
```

	col1	col2	col3
WI	414	715	262
MN	651	612	218

```
print(areacodes.rank(axis='columns'))
                          #
```

	col1	col2	col3
WI	2.0	3.0	1.0
MN	3.0	2.0	1.0

```
print(areacodes.rank(axis='rows'))
                          #
```

	col1	col2	col3
WI	1.0	2.0	2.0
MN	2.0	1.0	1.0

2.C.5 Merging DataFrames

- The **`merge()`** function can be used to merge two or more DataFrames based on multiple common columns
  - ◊ Example:

```
inventory = pd.DataFrame({
    "prod_num": [658, 2476, 3911],
    "quantity": [120, 54, 98]})

sales_det = pd.DataFrame({
    "prod_num": [658, 2476, 658, 2476, 3911, 3911],
    "price": [7.89, 5.49, 7.89, 5.49, 6.25, 6.25]})
sales = sales_det.groupby(['prod_num']).mean()

merged_df = pd.merge(inventory, sales, on=['prod_num'])
print(merged_df)
```

#	prod_num	quantity	price
0	658	120	7.89
1	2476	54	5.49
2	3911	98	6.25

    - * How does the result look different if merging `sales_det` instead of `sales` with `inventory`?

Module 3

Matplotlib

Section 3.A

Matplotlib Basics

3.A.1 Getting started with Matplotlib

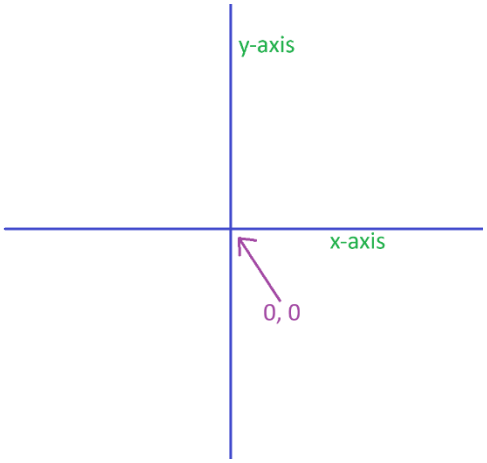
- Matplotlib is a graph plotting library frequently used in Python for creating basic visualizations
  - ◊ Some newer visualization libraries like Seaborn are built on Matplotlib
- Most of Matplotlib's utilities can be found in the pyplot module
  - ◊ This means that usually just that portion of Matplotlib is imported:

```
import matplotlib.pyplot as plt
```

- Creating visualizations is known as **plotting**
- The elements of a Matplotlib visualization are formally known as **artists** (including lines, labels, point markers, etc.)

3.A.2 A quick refresher on graphing basics

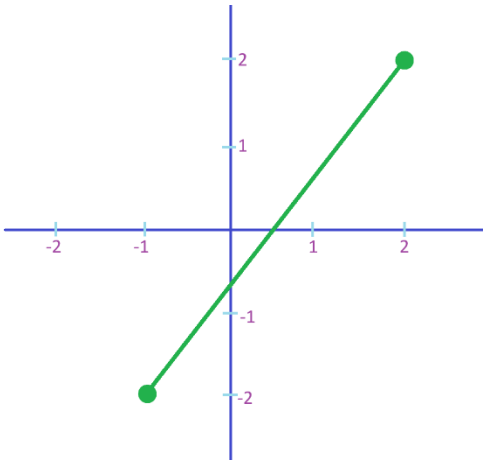
- In math, a graph has an **x-axis** (horizontal) and **y-axis** (vertical)
  - ◊ Example:



- **Points** are plotted on the graph using (x, y) coordinates, where x is the position along the x axis, and y is the position along the y axis

- **Lines** are defined based on connecting / passing through points
  - ◊ Example:

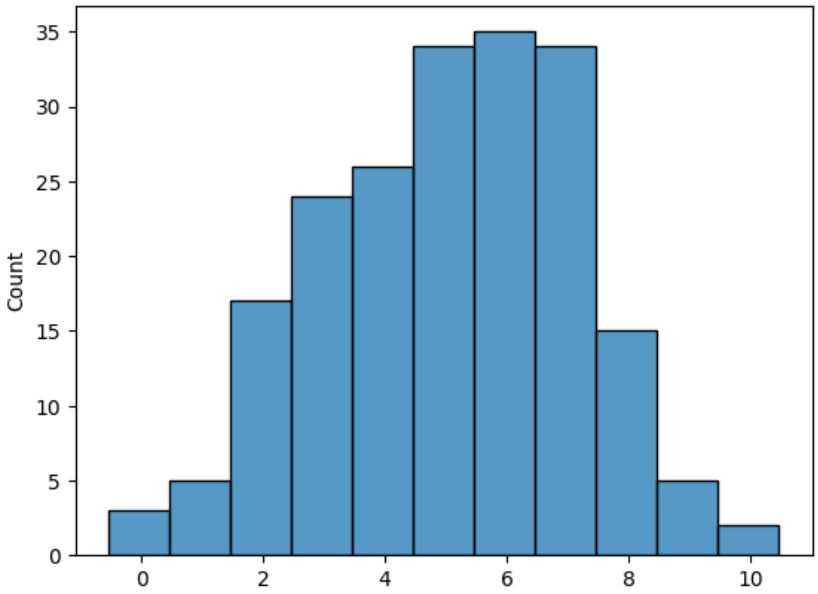
    Plotting (-2, -1) to (2, 2)



3.A.3 Continuing graph basics

- For styles of graph or chart designed for data visualization, even when we're not making line graphs, we still refer to the horizontal axis as the x-axis, and the vertical axis as the y-axis

- For example, a histogram groups similar values in a distribution – no line plotting, but still graphing data against an x-axis and y-axis:

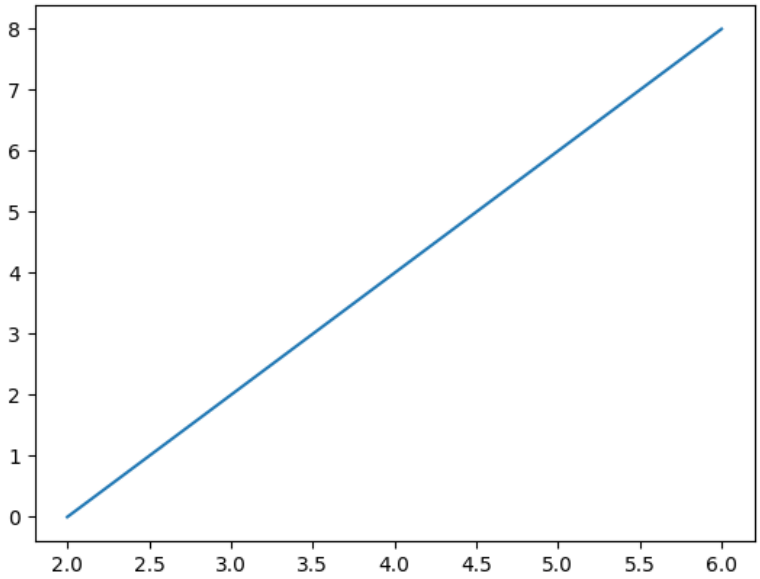


3.A.4 A simple line graph with Matplotlib

- The **`plot()`** function is used to plot points on a graph, and by default draws a line from point to point
  - ◊ The first parameter (*scalex*) is an array containing the points on the x-axis
  - ◊ The second parameter (*scaley*) is an array with the points on the y-axis
  - ◊ For example, to plot a line from point (2, 0) to point (6, 8), we have to pass two arrays to the plot function: [2, 6] and [0, 8]

```
xpoints = np.array([2, 6])
ypoints = np.array([0, 8])
plt.plot(xpoints, ypoints)
plt.show()
```

Result:



  - ◊ You can also specify only points on the y-axis, in which case the x-axis values will default to 0, 1, 2, etc. (corresponding to number of y points)

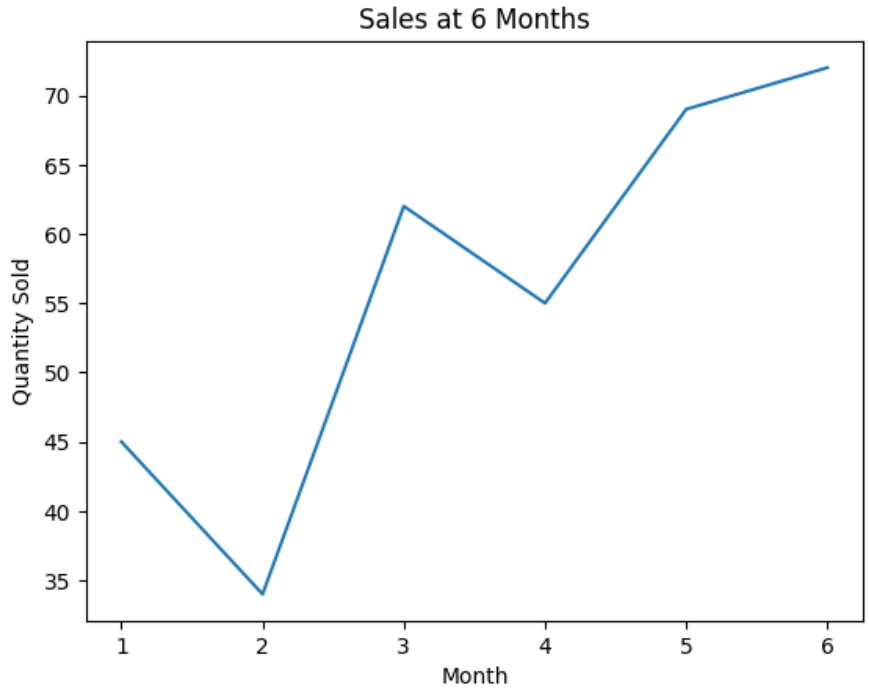
- The **`show()`** function displays the plot (sort of like `print()`)

3.A.5 Plotting data from a DataFrame

- Matplotlib's plot() function can plot data from a pandas DataFrame
  - ◊ Example:

```
import matplotlib.pyplot as plt
import pandas as pd

sales = pd.DataFrame({
    "month": [1, 2, 3, 4, 5, 6],
    "qty_sold": [45, 34, 62, 55, 69, 72]})

plt.title("Sales at 6 Months")
plt.xlabel("Month")
plt.ylabel("Quantity Sold")

plt.plot(sales['month'], sales['qty_sold'])
plt.show()
```

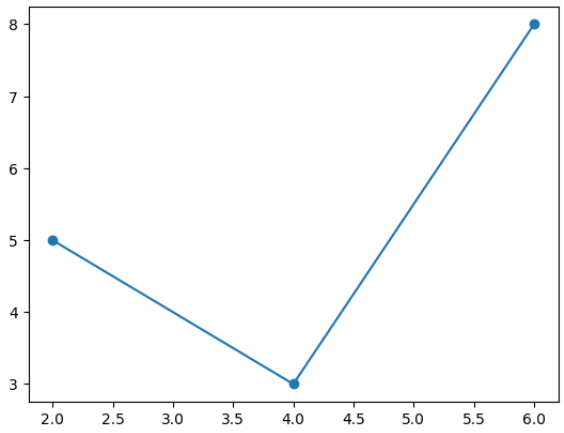
Result:



3.A.6 Plotting markers

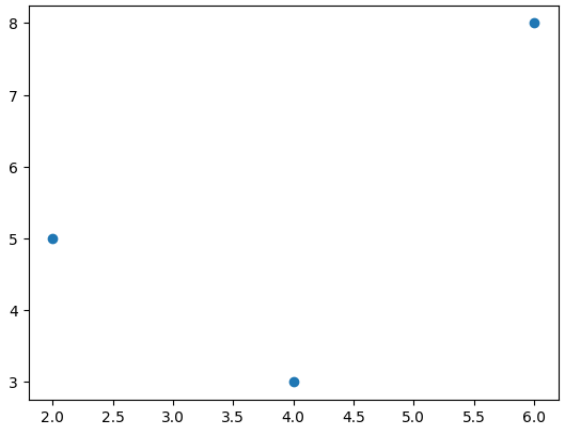
- To plot **markers** for the points along with the line, use the optional parameter **`marker='o'`**
  - ◊ Example:

```
plt.plot([2, 4, 6], [5, 3, 8], marker='o')
```



- To plot only the markers (not a line), you can include parameter **`'o'`** *without* the `marker` keyword
  - ◊ Example:

```
plt.plot([2, 4, 6], [5, 3, 8], 'o')
```



3.A.7 Styling markers

- There are many styles of marker to choose from:

'o'	Circle
'*'	Star
'.'	Point
','	Pixel
'x'	X
'X'	X (filled)
'+'	Plus
'P'	Plus (filled)
's'	Square
'D'	Diamond
'd'	Diamond (thin)
'p'	Pentagon
'H'	Hexagon
'h'	Hexagon
'v'	Triangle Down
'^'	Triangle Up
'<'	Triangle Left
'>'	Triangle Right
'1'	Tri Down
'2'	Tri Up
'3'	Tri Left
'4'	Tri Right
'|'	Vline
'_'	Hline

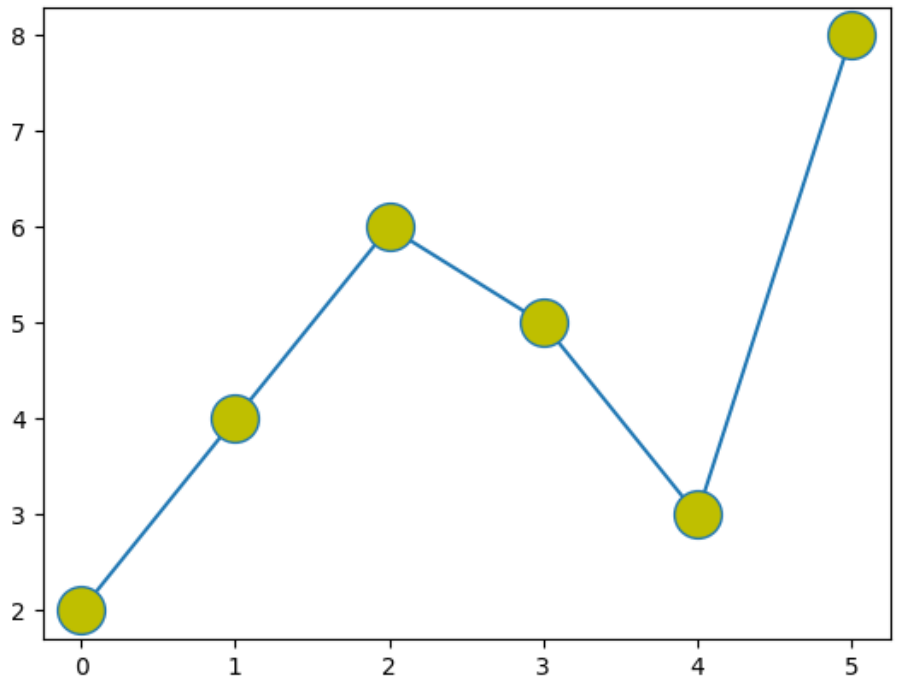
3.A.8 Styling markers, size and color

- To change the size of markers, use the keyword argument **`ms=`** (`markersize=` also works)

- You can also change the color of the markers' outline or fill:
  - ◊ **`mec=`** (or `markeredgecolor=`) alters the outline
  - ◊ **`mfc=`** (or `markerfacecolor=`) sets the fill color / face color
    - * A color reference chart is included on the next page

- For example:
  - ◊ Changing both marker size and face color:

```
plt.plot([2, 4, 6, 5, 3, 8], marker='o', ms=20, mfc='y')
```



3.A.9 Using `fmt` parameter for styling

- You can also use the **shortcut string notation** (**`fmt`**) parameter
  - ◊ The fmt parameter uses the syntax: `'marker|line|color'`
  - ◊ Line formatting can use the following styles:

`'-'`	Solid line
`':'`	Dotted line
`'--'`	Dashed line
`'-.'`	Dashed/dotted line

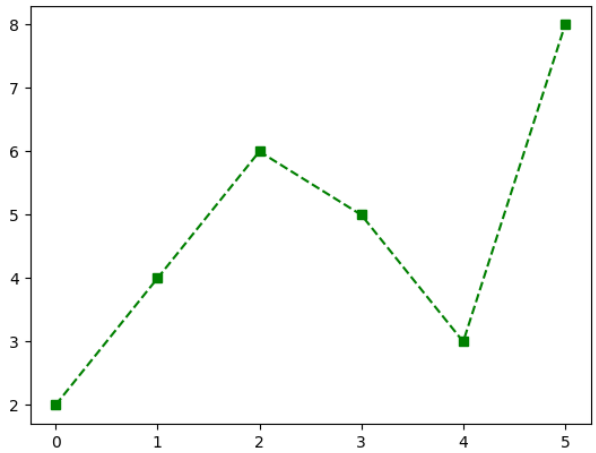
  - ◊ Colors include the below, or use hexadecimal or HTML color names:

`'r'`	Red
`'g'`	Green
`'b'`	Blue
`'c'`	Cyan
`'m'`	Magenta
`'y'`	Yellow
`'k'`	Black
`'w'`	White

For reference list of recognized HTML color names, see: *https://www.w3schools.com/colors/colors_hex.asp*

  - ◊ Example:

```
plt.plot([2, 4, 6, 5, 3, 8], 's--g')
```



3.A.10 Styling lines

- You can use the keyword argument **`ls=`**, or `linestyle=`, to change the style of the plotted line
  - ◊ In addition to the short syntax listed on the previous page, you can also use the following reader-friendly descriptions:

`'solid'`	(default), shortened as `'-'`
`'dotted'`	`':'`
`'dashed'`	`'--'`
`'dashdot'`	`'-.'`
`'None'`	`''` or `' '` (empty string)

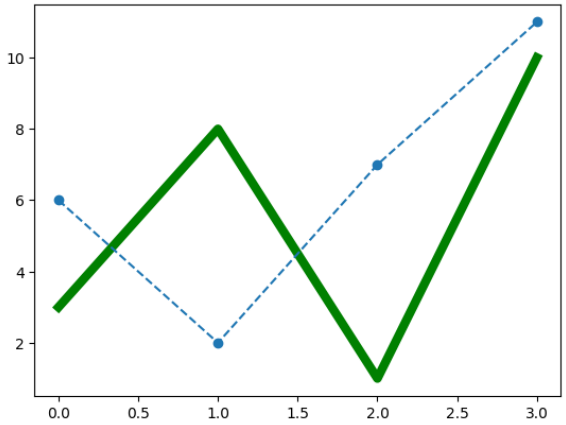
- To change the line color, use **`c=`** or `color=`
  - ◊ Again, you can use any color abbreviation from the previous page, or any hexadecimal color, or any color name from the recognized web-standard HTML color values

- To change the thickness of the line, use **`lw=`** or `linewidth=`

- You can also plot multiple lines with multiple plt.plot() functions

Example:

```
a = np.array([3, 8, 1, 10])
b = np.array([6, 2, 7, 11])
plt.plot(a, lw=6, c='g')
plt.plot(b, ls='--', marker='o')
plt.show()
```



3.A.11 Figure size

- Before drawing any plot, you can optionally set the figure size using `plt.figure()` with the **`figsize=`** parameter

- `figsize` takes a tuple of (width, height) in inches:

  ◊ Example:

  ```
  plt.figure(figsize=(10, 6))
  ```

- Larger values produce bigger charts; the default is approximately (6.4, 4.8)

- This is especially useful when a chart has long labels, many bars, or needs to fit a specific layout in a report

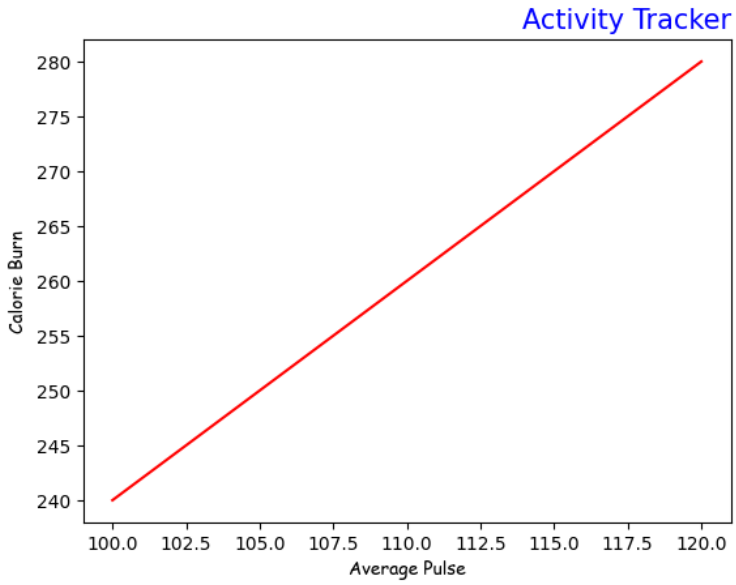
Section 3.B

Continuing with Matplotlib

3.B.1 Using labels

- To set labels in a plot:
  - ◊ For each axis, use `plt.xlabel()` and `plt.ylabel()`
  - ◊ Use `plt.title()` to set a plot title, with optional loc= parameter to set position ('left', 'right', or 'center' [default])
- For each of the above, font properties can also be set
  - ◊ Parameter **`fontdict=`** accepts a dictionary of font properties such as: `{'family':'serif','color':'blue','size':20}`
    - * Font family options include the generic families `'serif', 'sans-serif', 'monospace', 'fantasy' or 'cursive'`
    - * For documentation on font family customization, see: https://matplotlib.org/stable/gallery/text_labels_and_annotations/font_family_rc.html
- Example:

```
x = np.array([100, 110, 120])
y = np.array([240, 260, 280])
font1 = {'color':'darkblue',
         'size':15}
font2 = {'family':'cursive'}
plt.title("Activity Tracker",
          fontdict = font1,
          loc='right')
plt.xlabel("Average Pulse",
           fontdict = font2)
plt.ylabel("Calorie Burn",
           fontdict = font2)
plt.plot(x, y, c='red')
plt.show()
```



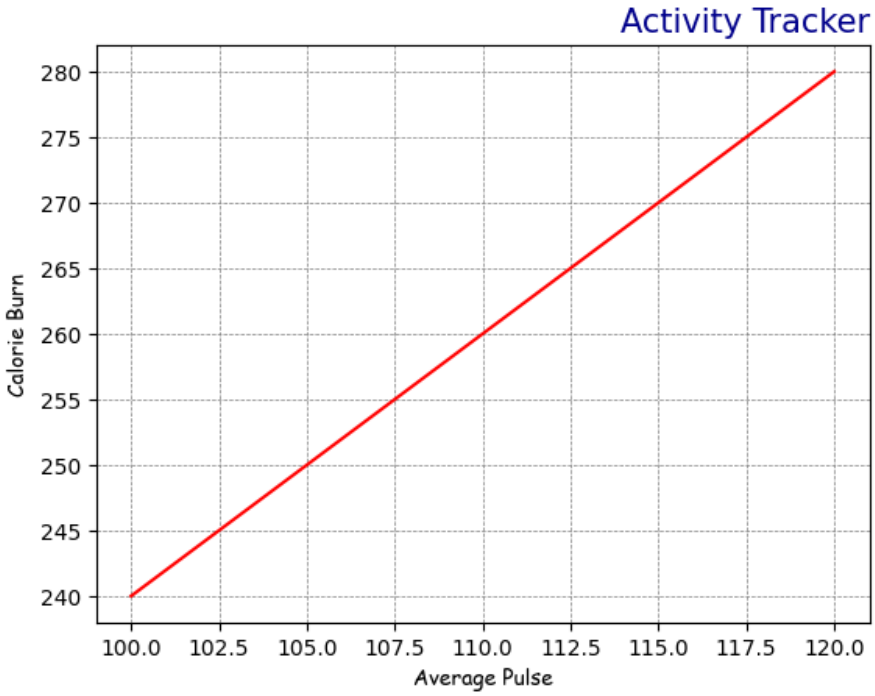
3.B.2 Adding a grid

- The **`grid()`** function can be used to add grid lines to the plot
  - ◊ You can use the **`axis=`** parameter to specify which grid lines to display
  - ◊ You can also set the line properties of the grid, for example:
    `grid(color = 'red', linestyle = '--', linewidth = .5)`
  - ◊ Example:

    [same starting variables for `x`, `y`, `font1`, `font2` as previous]

```
...
plt.title("Activity Tracker", fontdict=font1, loc='right')
plt.xlabel("Average Pulse", fontdict = font2)
plt.ylabel("Calorie Burn", fontdict = font2)

plt.plot(x, y, c='red')
plt.grid(color = 'gray', linestyle = '--', linewidth = .5)
plt.show()
```

    Result:



3.B.3 Bar charts

- **Bar charts** are useful for comparing categorical data, where one axis counts numerical data and the other shows categories/names

- The **`bar()`** function can be used to draw a bar graph – or to display bars horizontally, use the **`barh()`** function

  ◊ The first parameter takes an array of categories, the second an array of number values

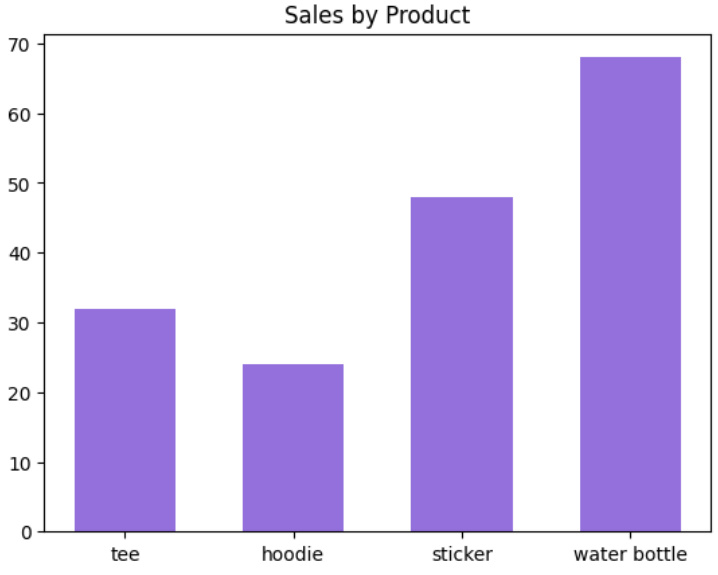
- In addition to label customization, can also specify:

  ◊ Bar color using `color=`

  ◊ Bar width using `width=` (or for `barh()`, using `height=`)

- Example:

```
by_product = pd.DataFrame(
    {"item": ['tee',
              'hoodie',
              'sticker',
              'water bottle'],
    "qty_sold": [32,24,48,68]})

plt.title("Sales by Product")

plt.bar(by_product['item'],
        by_product['qty_sold'],
        color='mediumpurple',
        width=.6)
plt.show()
```



3.B.4 Scatter charts

- A **scatter chart** plots points using numerical data on both the x-axis and y-axis, and is especially used to look for trends or clusters

- The **`scatter()`** function is used to draw a scatter plot
  - ◊ This function takes parameters of two numerical arrays of equal length, one for the x-axis and one for the y-axis
  - ◊ To compare plots, two or more `scatter()` functions can be called for the same figure

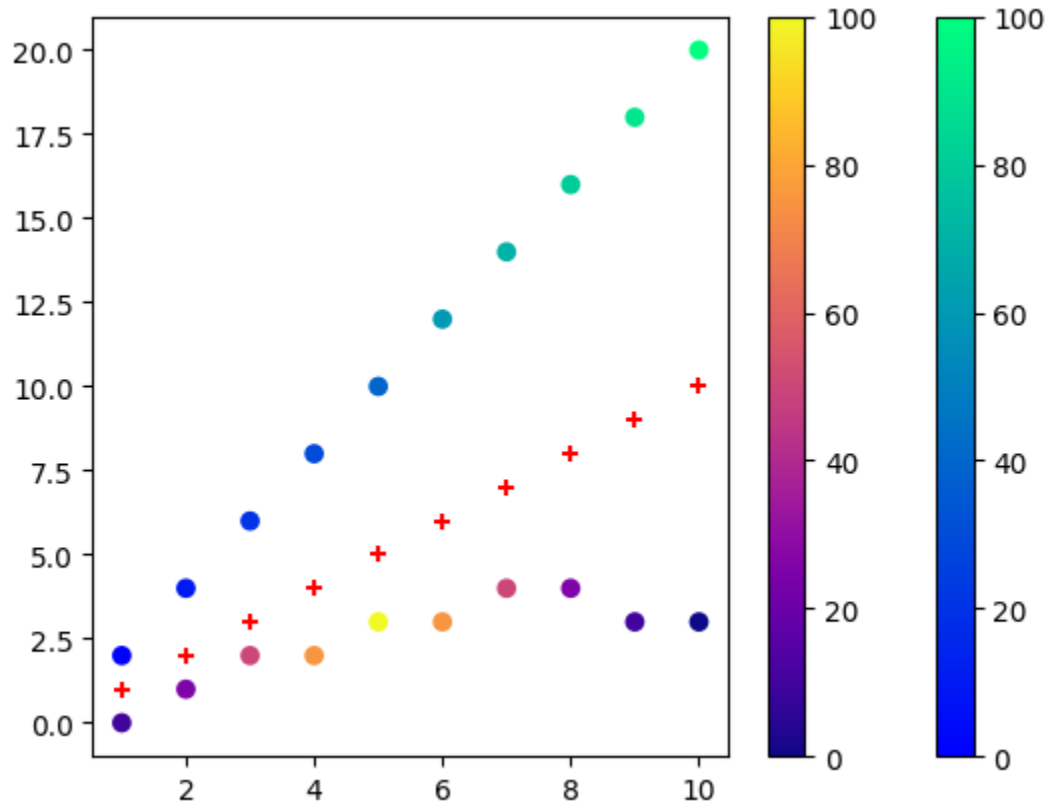
- Using the **`color=`** or **`c=`** keyword argument, a color can be specified for all dots in the plot…
  - ◊ … *or* each dot can be assigned its own color using an array with c=
  - ◊ … *or* to apply "heat map" coloring, you can combine c= (as an array of values up to 100) with **`cmap=`** and the name of a recognized color map (see: https://matplotlib.org/stable/users/explain/colors/colormaps.html)

3.B.5 Scatter charts (continued)

◊ Example:

```
a = np.array([1,2,3,4,5,6,7,8,9,10])
b = np.array([2,4,6,8,10,12,14,16,18,20])
c = np.array([0,1,2,2,3,3,4,4,3,3])
colors1 = np.array([0,10,20,30,40,60,70,80,90,100])
colors2 = np.array([10,25,50,75,100,75,50,25,90,0])

plt.scatter(a, b, c=colors1, cmap='winter')
plt.colorbar()
plt.scatter(a,c, c=colors2, cmap='plasma')
plt.colorbar()
plt.scatter(a,a, c='red', marker='+')

plt.show()
```

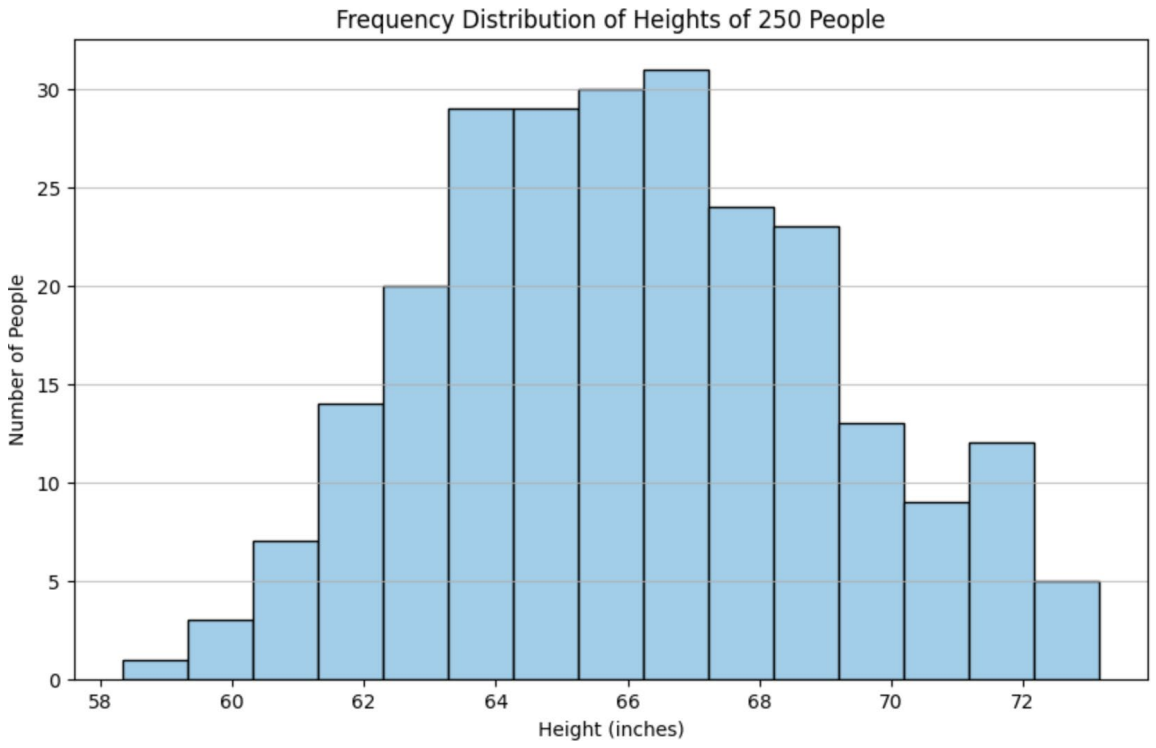
Result:



3.B.6 Histograms

- A histogram is a graph showing frequency distributions by interval
  - ◊ Example: Say you ask for the height of 250 people, you might end up with a histogram like this:

```
heights = np.random.normal(loc=66, scale=3, size=250)

plt.figure(figsize=(10, 6))
plt.hist(heights, bins=15, color='skyblue',
edgecolor='black')

plt.title('Frequency Distribution of Heights of 250
People')
plt.xlabel('Height (inches)')
plt.ylabel('Number of People')
plt.grid(axis='y', alpha=0.75)
plt.show()
```

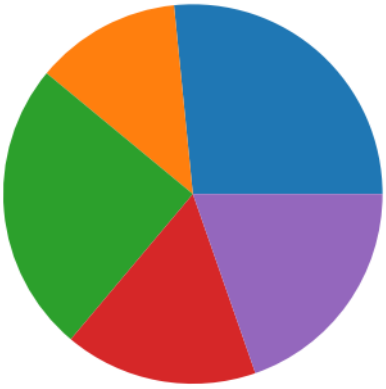


3.B.7 Pie charts

- The pie() function can be used to draw a pie chart
  - ◊ Example:

```
p = np.array([92,43,86,57,68])
plt.pie(p)
plt.show()
```

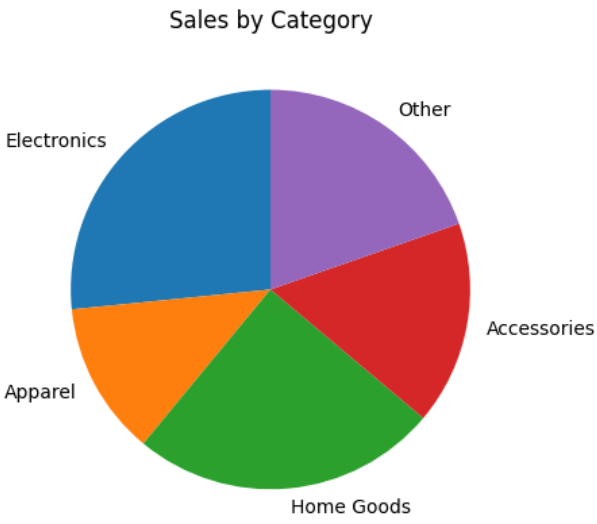
Result:



- Optional parameters include `labels`, `startangle`, `explode`, `shadow`, and `colors`

```
categories = [
    'Electronics', 'Apparel',
    'Home Goods', 'Accessories',
    'Other']
sales = [92, 43, 86, 57, 68]

plt.pie(
    sales,
    labels=categories,
    startangle=90)
plt.title('Sales by Category')
plt.show()
```

Result:

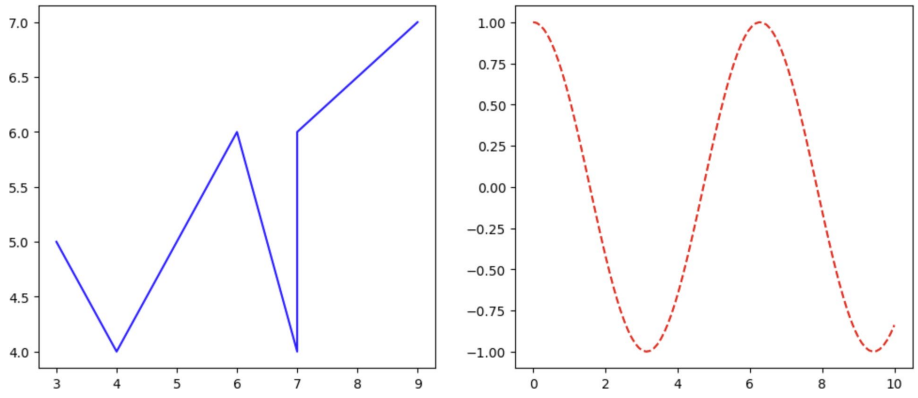


- A legend can also be added using `legend()` with optional header

3.B.8 Subplots

- Subplots allow you to draw multiple plots within one figure
  - ◊ The **`plt.subplots()`** function returns two objects:
    - `fig` (Figure object): Represents the entire figure or plotting area – the container that holds all plot elements
    - `axes` (Axes objects): An array (or a single object if there's only one subplot) of Axes objects, which are the individual plots within the figure where data is plotted
    - Example 1:

```
x1 = [3,4,6,7,7,9]
y1 = [5,4,6,4,6,7]

x2 = np.linspace(0, 10, 100)
y2 = np.cos(x2)

fig, axes = plt.subplots(1, 2, figsize=(12, 5))
                                    # unpacks returned tuple into 2 variables

axes[0].plot(x1, y1, color='blue', linestyle='-')
axes[1].plot(x2, y2, color='red', linestyle='--')
plt.show()
```



3.B.9 Saving a figure

- To save a chart to a file instead of (or in addition to) displaying it, use **`plt.savefig()`**
  - ◊ Call before `plt.show()`, passing a filename with the desired extension
  - ◊ Example:

```
plt.savefig('awesome_chart.png')
```

- Common save formats: .png (images and reports), .jpg, .pdf, .svg
- Use the optional `dpi=` parameter to control resolution
  - ◊ `dpi=150` or `dpi=300` is typical for reports:

```
plt.savefig('my_chart.png', dpi=150)
```

Module 4

scikit-learn

Section 4.A

Introduction to Predictive Analysis

4.A.1 What is machine learning?

- **Machine learning** is a method of teaching a computer to find patterns in data and use those patterns to make predictions or decisions, without being explicitly programmed with rules

- There are two broad categories:
  - ◊ **Supervised learning**: you provide labeled examples (inputs + known answers), and the model learns to predict the answer for new inputs
  - ◊ **Unsupervised learning**: you provide data without labels, and the model finds structure or groupings on its own

- Machine learning is a tool to help answer the question "What can I predict from this data?"

- This module focuses on using scikit-learn to predict a continuous numeric outcome, known as regression

4.A.2 The estimator API

- Every model in scikit-learn follows the same three-step interface, regardless of which algorithm you use. This consistent pattern is called the estimator API:

Method	What it does
`.fit(X, y)`	Trains the model on your data
`.predict(X)`	Uses the trained model to generate predictions
`.score(X, y)`	Evaluates model accuracy (returns R^2 for regression models)

- This means that once you learn linear regression using this pattern, switching to a different model type (if you go further in machine learning) requires almost no new syntax, just a different import and class name

4.A.3 Preparing your data

- scikit-learn models expect data in a specific format:
  - ◊ **Feature matrix X**: a 2D array or DataFrame of input columns (the things you are using to make a prediction)
  - ◊ **Target vector y**: a 1D array or Series of the values you want to predict
- Before training, it's good practice to split your data into two sets:
  - ◊ **Training set**: the data the model learns from
  - ◊ **Test set**: data held back to evaluate how well the model performs on new, unseen examples
  - ◊ To split, use `train_test_split()` from `sklearn.model_selection`:

```
from sklearn.model_selection import train_test_split

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)
```

    * `test_size=0.2` reserves 20% of the data for testing; 80% goes to training
    * `random_state=42` sets a seed so the split is reproducible (you get the same split every time you run the code)
- Why split your data?
  - ◊ A model evaluated on its own training data will always look accurate because it has already "seen" those answers
  - ◊ Testing on held-back data gives you an honest measure of how well the model generalizes to new data

Section 4.B

Linear Regression

4.B.1 When to use linear regression

- Use linear regression when:
  - ◊ The outcome you want to predict is a continuous number (a sales total, a temperature, a price)
  - ◊ You expect a roughly linear relationship between your inputs and that outcome (as one goes up, the other tends to go up or down proportionally)
- You have already seen this concept illustrated elsewhere:
  - ◊ Excel's trendline feature draws a line through a scatter plot that minimizes the distance between the line and the data points
  - ◊ Linear regression does exactly the same thing, but programmatically and with the ability to use multiple input variables at once
- Examples of questions linear regression can answer:
  - ◊ "Given the square footage and number of bedrooms, what should this house sell for?"
  - ◊ "Given last month's ad spend, what will this month's revenue be?"
  - ◊ "Given a student's hours of study, what score can we predict?"

4.B.2 Building and fitting a model

- Open the sample script **linear_regression_example.py** in your course material:

```
import numpy as np
import pandas as pd
from sklearn.linear_model import LinearRegression
from sklearn.model_selection import train_test_split

# Sample dataset: hours studied vs. exam score
data = {
    'hours_studied': [1, 2, 3, 4, 5, 6, 7, 8, 9, 10],
    'exam_score':    [52, 55, 60, 65, 68, 72, 75, 80, 85, 90]
}
df = pd.DataFrame(data)

# Step 1: Define features (X) and target (y)
X = df[['hours_studied']]   # 2D: double brackets give a DataFrame
y = df['exam_score']        # 1D: single brackets give a Series

# Step 2: Split into train and test sets
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

# Step 3: Create the model and fit it to training data
model = LinearRegression()
model.fit(X_train, y_train)

print('Model trained successfully.')
```

- Notice that X uses double brackets [['hours_studied']] — this keeps X as a DataFrame (2D), which is what scikit-learn expects
  - ◊ Single brackets would give a Series (1D), which would cause an error

4.B.3 Making predictions

- Modify the sample script to make some predictions:

```
# Predict exam scores for the test set
y_pred = model.predict(X_test)

print('Actual scores:   ', list(y_test))
print('Predicted scores:', list(y_pred.round(1)))

# Inspect what the model learned
print(f'\nSlope (coefficient):  {model.coef_[0]:.2f}')
print(f'Intercept:            {model.intercept_:.2f}')
```

- What is the code doing here?
  - ◊ `model.coef_` is the slope(s) of the regression line; here it tells you: "for each additional hour studied, the predicted score increases by this many points"
    - * When you have a single feature (e.g. hours studied), model.coef_ returns a single slope value
    - * When you have multiple features (e.g., hours studied, hours slept, and practice tests taken), model.coef_ returns an array with one slope value per feature
  - ◊ `model.intercept_` is the predicted score when all inputs equal zero (often not meaningful on its own, but part of the line equation: score = slope × hours + intercept)
  - ◊ These two values define the line that the model fit to your training data (the same line Excel would draw as a trendline)

4.B.4 Evaluating the model

- Continuing with the sample script:

```
r_squared = model.score(X_test, y_test)
print(f'R² score: {r_squared:.3f}')
```

- **R^2 (R-squared)** measures how much of the variation in your target variable is explained by your model:
  - ◊ **$R^2 = 1.0$** → perfect predictions; the model explains all variation in the data
  - ◊ **$R^2 = 0.0$** → the model has no predictive value; it performs no better than always predicting the average
  - ◊ **$R^2 = 0.85$** → the model explains 85% of the variation; 15% remains unexplained by the input features

- R^2 alone doesn't tell you whether a model is good enough to use because it depends on the context
  - ◊ For example, an R^2 of 0.7 might be excellent for predicting social behavior but poor for predicting a physical measurement

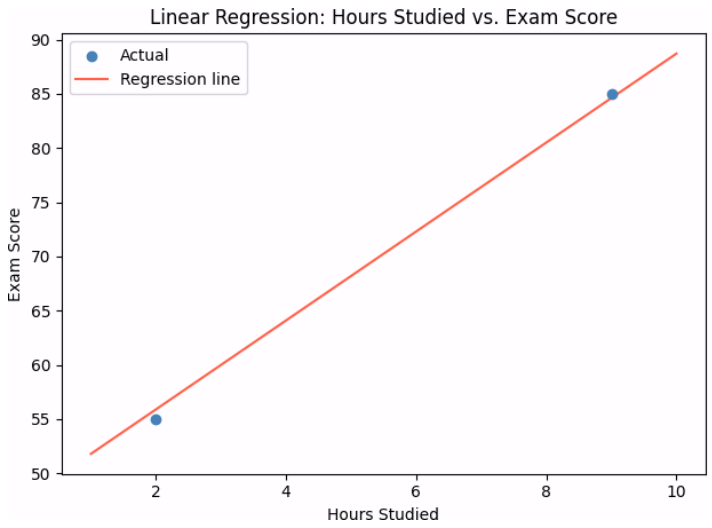
4.B.5 Visualizing actual vs. predicted

- Continuing with the sample script:

```
import matplotlib.pyplot as plt

# Scatter plot: actual test data points
plt.scatter(X_test, y_test, color='steelblue', label='Actual', zorder=3)

# Line: model predictions across the full range of X
x_range = np.linspace(X.min(), X.max(), 100).reshape(-1, 1)
y_line = model.predict(x_range)
plt.plot(x_range, y_line, color='tomato', label='Regression line')

plt.xlabel('Hours Studied')
plt.ylabel('Exam Score')
plt.title('Linear Regression: Hours Studied vs. Exam Score')
plt.legend()
plt.tight_layout() # adjusts spacing so labels and titles don't overlap
plt.show()
```

- The scatter plot shows the actual test data points (what really happened)

- The regression line shows what the model predicts across the entire range of input values



- This chart ties together everything from this week: a pandas DataFrame as the data source, NumPy to generate the prediction range, and Matplotlib to produce the final visualization

4.B.6 Week 7 Wrap-Up

- You now have a grounding in four foundational libraries for data analysis in Python:
  - ◊ NumPy gives you fast, flexible numerical computing
  - ◊ pandas lets you load, clean, and reshape real-world datasets
  - ◊ Matplotlib turns that data into charts and visualizations
  - ◊ scikit-learn is where data analysis meets machine learning, giving you the tools to answer "what can I predict from this data?"
- Together these libraries form the Python toolkit that professional data analysts use to go from raw data to insight

End of Week 7 Student Guide